

EPIC 06 — Task 04: Redis Caching

1. Cache Read
2. Cache Write
3. Cache Expiration
4. Cache Invalidation
5. Measure API performance before and after caching

We can implement this in the **Product Service** using Spring Boot + Redis.

Step 1 — Add Redis Dependency

If you are using Maven, open your pom.xml and add:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

Step 2 — Configure Redis

Open:

src/main/resources/application.properties

Add:

```
spring.data.redis.host=localhost
spring.data.redis.port=6379
```

Redis will run on:

localhost:6379

Step 3 — Enable Caching

In your main Spring Boot class:

```
@SpringBootApplication
@EnableCaching
public class ProductServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```

Import:

```
import org.springframework.cache.annotation.EnableCaching;
```

Step 4 — Cache Read

Open your ProductService.java.

Use @Cacheable:

```
@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    @Cacheable(value = "products", key = "#id")
    public Product getProductById(Long id) {

        System.out.println("Fetching product from database...");

        return productRepository.findById(id)
    }
}
```

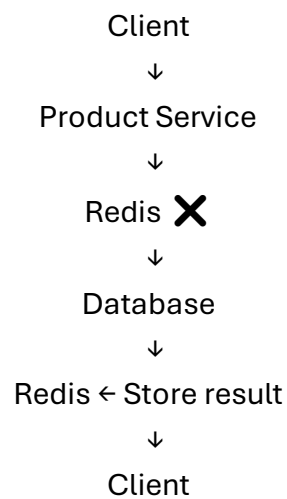
```
        .orElseThrow(() ->
            new RuntimeException("Product not found"));
    }
}
```

Import:

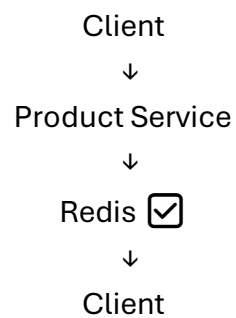
```
import org.springframework.cache.annotation.Cacheable;
```

How it works

First request:



Second request:



The database does not need to be queried again.

Step 5 — Cache Write

When a product is created or updated, we can update the cache.

```
@CachePut(value = "products", key = "#product.id")
public Product updateProduct(Product product) {

    return productRepository.save(product);
}
```

Import:

```
import org.springframework.cache.annotation.CachePut;
```

This makes sure the cached product contains the latest data.

Step 6 — Cache Invalidation

When a product is deleted, remove it from Redis.

```
@CacheEvict(value = "products", key = "#id")
public void deleteProduct(Long id) {

    productRepository.deleteById(id);
}
```

Import:

```
import org.springframework.cache.annotation.CacheEvict;
```

Flow:

Delete Product

↓

Database → Delete

↓

Redis → Remove Cache

This prevents stale data from remaining in the cache.

Step 7 — Cache Expiration

Configure an expiration time, so cached data automatically expires.

For example, use a **10-minute TTL**.

Create:

RedisConfig.java

@Configuration

```
public class RedisConfig {
```

```
    @Bean
```

```
    public RedisCacheManager cacheManager(  
        RedisConnectionFactory connectionFactory) {
```

```
        RedisCacheConfiguration config =
```

```
            RedisCacheConfiguration.defaultCacheConfig()  
                .entryTtl(Duration.ofMinutes(10))  
                .disableCachingNullValues();
```

```
        return RedisCacheManager.builder(connectionFactory)  
            .cacheDefaults(config)  
            .build();
```

```
    }
```

```
}
```

Imports:

```
import java.time.Duration;
```

```
import org.springframework.cache.annotation.EnableCaching;
```

```
import org.springframework.context.annotation.Bean;
```

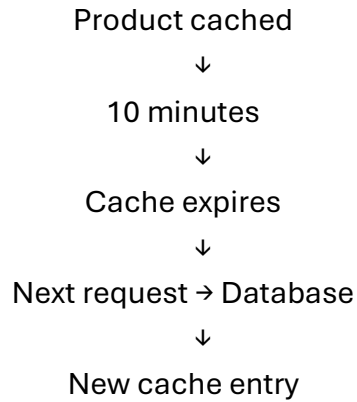
```
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.data.redis.cache.RedisCacheConfiguration;
```

```
import org.springframework.data.redis.cache.RedisCacheManager;
```

```
import org.springframework.data.redis.connection.RedisConnectionFactory;
```

Now:



Step 8 — Complete Product Service

Your service can look like this:

```
@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    // Cache Read
    @Cacheable(value = "products", key = "#id")
    public Product getProductById(Long id) {

        return productRepository.findById(id)
            .orElseThrow(() ->
                new RuntimeException("Product not found"));
    }
}
```

```

// Cache Write
@CachePut(value = "products", key = "#product.id")
public Product updateProduct(Product product) {

    return productRepository.save(product);
}

// Cache Invalidation
@CacheEvict(value = "products", key = "#id")
public void deleteProduct(Long id) {

    productRepository.deleteById(id);
}
}

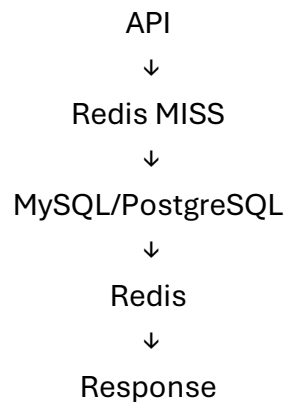
```

Step 9 — Test Cache Read

Call:

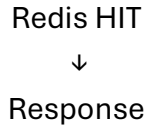
GET /products/1

First request



Second request





You should see a significant reduction in database calls for frequently requested products.

Step 10 — Measure Performance

Without Redis

Test the API several times and record:

Request 1 → 200 ms

Request 2 → 180 ms

Request 3 → 190 ms

Average → 190 ms

With Redis

Request 1 → 190 ms

Request 2 → 20 ms

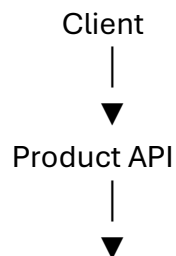
Request 3 → 15 ms

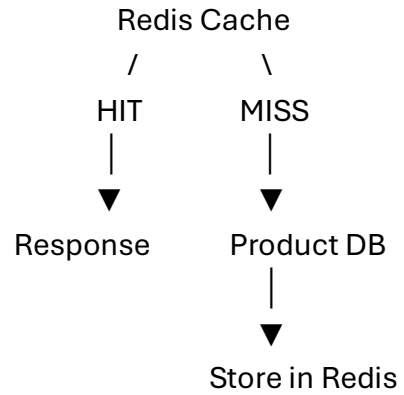
Average → 75 ms

These numbers are **examples**. Use the actual response times from your application.

You can test using **Postman** or another API testing tool.

Final Architecture:





Requirement

Cache Read
Cache Write
Cache Expiration
Cache Invalidation
Performance
Technology

Implementation

@Cacheable
@CachePut
10-minute TTL
@CacheEvict
Compare API response times
Spring Boot + Redis

Task 04 is complete.